

Technical Spec: Organizational Teams Scheduling System

1. Overview

A Calendly-style platform where an external visitor requests a meeting, an admin checks staff availability across the org, books a Microsoft Teams meeting with a chosen staff member, and can later reschedule, cancel, or reassign (swap host) that meeting — all via Microsoft Graph API.

2. Azure / Microsoft Entra ID Setup

A. App Registration

1. Azure Portal → Microsoft Entra ID → App Registrations → New registration (single-tenant, since this only needs to work within your org).
2. Authentication: generate a **Client Secret** (or, preferably for production, a **certificate** — more secure and doesn't need periodic rotation like a secret does). Store `Tenant ID`, `Client ID`, and secret/cert in a secrets manager (e.g. Azure Key Vault), not in code or plain config.
3. API Permissions → Microsoft Graph → **Application permissions**:
 - `User.Read.All` — list staff members.
 - `Calendars.ReadWrite` — create/update/delete events on any staff calendar.
 - `OnlineMeetings.ReadWrite.All` — generate Teams join links.
4. Click **Grant admin consent** (Global Admin only).

B. Application Access Policy — REQUIRED, not optional

This is the step most likely to get missed and cause silent failures.

`OnlineMeetings.ReadWrite.All` on its own is **not enough** to create a Teams meeting on behalf of another user under app-only auth. Microsoft requires a separate **Application Access Policy**, configured via Teams/Skype for Business PowerShell, that explicitly authorizes your app's Client ID to act on behalf of specific users (or a group).

```
# Connect as a Teams/SfB admin
Connect-MicrosoftTeams

# Create a policy scoped to your app
New-CsApplicationAccessPolicy -Identity "OrgSchedulerPolicy" `
  -AppIds "<your-client-id>" -Description "Allows scheduler app to create meetings"

# Grant it to a user, or apply globally if you want it to cover all staff
Grant-CsApplicationAccessPolicy -PolicyName "OrgSchedulerPolicy" -Identity <user-object-id>
# For org-wide: Grant-CsApplicationAccessPolicy -PolicyName "OrgSchedulerPolicy" -Global
```

Without this, event creation with `isOnlineMeeting: true` for a user other than the app itself will fail (commonly a 403/Forbidden or UnknownError). Allow up to ~30 minutes for the policy to propagate after applying it.

C. Access Token Request (Client Credentials Flow)

- **Endpoint:** POST `https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token`
- **Body (form-encoded):**
 - `client_id`
 - `client_secret` (or client assertion, if using a certificate)
 - `grant_type=client_credentials`
 - `scope=https://graph.microsoft.com/.default`

- **Caching:** token `expires_in` is typically ~3600s. Cache it and refresh proactively a few minutes before expiry — don't request a fresh token per call.

3. Core Workflow

Step 0: Visitor request intake (not in original scope, but implied by the ask)

The DB needs a **pending** state before a staff member is assigned — see §6. This is a front-end/API concern separate from Graph integration: a form or endpoint that captures visitor name, email, and desired time window, and creates a `Requested` record for the admin to act on.

Step 1: Check Availability

- **Endpoint:** `POST /users/{staff_email}/calendar/getSchedule`
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: application/json`, `Prefer: outlook.timezone="UTC"`
- **Request Body:**

```
{
  "schedules": ["staffA@yourorg.com", "staffB@yourorg.com"],
  "startTime": { "dateTime": "2026-09-05T09:00:00", "timeZone": "UTC" },
  "endTime": { "dateTime": "2026-09-05T17:00:00", "timeZone": "UTC" },
  "availabilityViewInterval": 30
}
```

- **Response:** `200 OK`

```
{
  "value": [
    {
      "scheduleId": "staffA@yourorg.com",
      "availabilityView": "0022000011",
      "scheduleItems": [
        {
          "status": "busy",
          "start": { "dateTime": "2026-09-05T10:00:00", "timeZone": "UTC" },
          "end": { "dateTime": "2026-09-05T11:00:00", "timeZone": "UTC" }
        }
      ],
      "workingHours": {
        "daysOfWeek": ["monday", "tuesday", "wednesday", "thursday", "friday"],
        "startTime": "09:00:00.0000000",
        "endTime": "17:00:00.0000000",
        "timeZone": { "name": "UTC" }
      }
    }
  ]
}
```

- `availabilityView` is a string of digits, one per interval: `0` = free · `1` = tentative · `2` = busy · `3` = out of office · `4` = working elsewhere. Handle all five in the admin UI, not just free/busy. `scheduleItems` gives the actual busy blocks if you need to show “why” a slot is unavailable.

Step 2: Create the Teams Meeting

- **Endpoint:** `POST /users/{staff_email}/events`
- **Headers:** `Authorization: Bearer {token}`, `Content-Type: application/json`, `Prefer: outlook.timezone="UTC"`
- **Request Body:**

```
{
  "subject": "Organization Introduction Meeting",
```

```

"start": { "dateTime": "2026-09-05T14:00:00", "timeZone": "UTC" },
"end": { "dateTime": "2026-09-05T14:30:00", "timeZone": "UTC" },
"isOnlineMeeting": true,
"onlineMeetingProvider": "teamsForBusiness",
"attendees": [
  { "emailAddress": { "address": "visitor@client.com" }, "type": "required" }
]
}

```

- **Response:** 201 Created — abridged to the fields you actually need:

```

{
  "id": "AAMkAGI2NGRkMTU5LWE5...",
  "subject": "Organization Introduction Meeting",
  "start": { "dateTime": "2026-09-05T14:00:00.0000000", "timeZone": "UTC" },
  "end": { "dateTime": "2026-09-05T14:30:00.0000000", "timeZone": "UTC" },
  "organizer": {
    "emailAddress": { "name": "Staff A", "address": "staffA@yourorg.com" }
  },
  "attendees": [
    { "emailAddress": { "address": "visitor@client.com" }, "type": "required", "status": {
"response": "none" } }
  ],
  "isOnlineMeeting": true,
  "onlineMeeting": {
    "joinUrl": "https://teams.microsoft.com/l/meetup-join/19%3ameeting_..."
  },
  "webLink": "https://outlook.office365.com/owa/?itemid=...&exvsurl=1&path=/calendar/item"
}

```

- The two fields you must persist: **id** (event ID — required for every later PATCH/DELETE/cancel call) and **onlineMeeting.joinUrl** (the link you send the visitor). **webLink** is useful if you also want a “view in Outlook” link.

Step 3: Reschedule

- **Endpoint:** PATCH /users/{staff_email}/events/{event_id}
- **Headers:** Authorization: Bearer {token}, Content-Type: application/json, Prefer: outlook.timezone="UTC"
- **Body:** send only the fields that change — typically **start** and **end**. Graph updates the existing event in place; the Teams **joinUrl** and event **id** stay the same.

```

{
  "start": { "dateTime": "2026-09-06T15:00:00", "timeZone": "UTC" },
  "end": { "dateTime": "2026-09-06T15:30:00", "timeZone": "UTC" }
}

```

- **Response:** 200 OK — the full updated event object (same shape as Step 2’s response), with **start/end** reflecting the new time and **id** and **onlineMeeting.joinUrl** unchanged:

```

{
  "id": "AAMkAGI2NGRkMTU5LWE5...",
  "subject": "Organization Introduction Meeting",
  "start": { "dateTime": "2026-09-06T15:00:00.0000000", "timeZone": "UTC" },
  "end": { "dateTime": "2026-09-06T15:30:00.0000000", "timeZone": "UTC" },
  "onlineMeeting": { "joinUrl": "https://teams.microsoft.com/l/meetup-join/19%3ameeting_..." },
  "lastModifiedDateTime": "2026-09-01T10:22:11Z"
}

```

All attendees (including the external visitor) are automatically sent a Graph-generated update notification — no separate email step needed, though you may still want your own confirmation email for branding. -

DB: update **requested_start** / **requested_end** (or a dedicated **scheduled_start** / **scheduled_end**) and set **status = Rescheduled**.

Step 4: Cancel / Delete

Two options depending on whether you want attendees notified:

- **Option A — Cancel (preferred):** POST `/users/{staff_email}/events/{event_id}/cancel` with an optional `comment` in the body. This sends a cancellation message to all attendees (the visitor gets a proper “meeting cancelled” notice in their calendar) and then removes the event.
 - **Headers:** Authorization: Bearer `{token}`, Content-Type: `application/json`
 - **Request Body:**

```
{ "comment": "Apologies, this meeting has been cancelled. We'll be in touch to reschedule." }
```
 - **Response:** 202 Accepted, empty body — treat this as success; there’s no payload to parse. The event is asynchronously removed from the organizer’s and attendees’ calendars.
- **Option B — Delete (no attendee notification):** DELETE `/users/{staff_email}/events/{event_id}`. Removes the event outright without sending a cancellation message — use only if you’re handling visitor notification yourself outside Graph (e.g. a custom email), otherwise the visitor’s calendar entry is orphaned with no explanation.
 - **Headers:** Authorization: Bearer `{token}`
 - **Response:** 204 No Content, empty body.
- **DB:** set `status = Cancelled`; keep the row (don’t hard-delete it) for audit/history purposes, and record `cancelled_at` / `cancelled_by`.

4. Swapping Hosts

Recommended: Approach B — Attendee Swap via Service Account, since it avoids regenerating and re-sending a new Teams link to the external visitor.

1. All meetings are created under a service account calendar (e.g. `scheduler@yourorg.com`); the assigned staff member is added as a required attendee, not the organizer.
2. To swap: PATCH `/users/{service_account}/events/{event_id}`.
 - **Headers:** Authorization: Bearer `{token}`, Content-Type: `application/json`
 - **Request Body:** send the full replacement `attendees` array — remove Person A, add Person B, keep the visitor:

```
{
  "attendees": [
    { "emailAddress": { "address": "staffB@yourorg.com" }, "type": "required" },
    { "emailAddress": { "address": "visitor@client.com" }, "type": "required" }
  ]
}
```

- **Response:** 200 OK with the updated event object; `attendees` now reflects Person B instead of Person A, while `id` and `onlineMeeting.joinUrl` are unchanged:

```
{
  "id": "AAMkAGI2NGRkMTU5LWE5...",
  "attendees": [
    { "emailAddress": { "address": "staffB@yourorg.com" }, "type": "required", "status": {
"response": "none" } },
    { "emailAddress": { "address": "visitor@client.com" }, "type": "required", "status": {
"response": "none" } }
  ],
  "onlineMeeting": { "joinUrl": "https://teams.microsoft.com/l/meetup-join/19%3ameeting_..." }
}
```

Graph automatically sends Person A a cancellation and Person B a new invite; the visitor gets an “updated attendees” notice, not a new join link.

3. Outcome: join URL and event ID stay the same; only the assigned attendee changes.
4. Trade-off: the staff member is an attendee, not the organizer, so they won’t have organizer-level

controls (lobby management, recording ownership, attendance reports). Flag this to the TL as a product decision — if organizer-level control per staff member is required, Approach A (delete-and-recreate on the staff member’s own calendar, generating a new link and notifying the visitor) is the alternative.

Action	Method	Endpoint	Notifies attendees?
Reschedule	PATCH	/users/{host}/events/{id}	Yes (automatic)
Cancel	POST	/users/{host}/events/{id}/cancel	Yes (with comment)
Delete	DELETE	/users/{host}/events/{id}	No
Swap host	PATCH	/users/{host}/events/{id} (attendees array)	Yes (automatic)
List staff	GET	/users?\$select=displayName,userPrincipalName	—

See §3 Steps 3–4 for full request/response detail on reschedule, cancel, and delete.

5. Error Handling & Reliability

- **429 Too Many Requests:** Graph throttles under load; respect the `Retry-After` header and back off/retry.
- **403 Forbidden:** usually means either admin consent or the Application Access Policy (§2B) hasn’t been granted/propagated yet.
- **404:** event already deleted/moved — handle gracefully in swap/cancel flows.
- Consider **Microsoft Graph change notifications (webhooks)** on calendar resources so that if a staff member edits/cancels the meeting directly in Outlook/Teams, your DB doesn’t go stale. Treat as a fast-follow, not blocking v1.

6. Database Schema

Field	Notes
visitor_email / visitor_name	for notifications
requested_start / requested_end	from intake, before a host is assigned
status	Requested → Scheduled → Rescheduled → Swapped → Cancelled (any state can move to Cancelled)
cancelled_at / cancelled_by	audit trail; keep the row rather than hard-deleting it
staff_email	current host/attendee
ms_event_id	Graph event ID, needed for all PATCH/DELETE calls
join_url	Teams link shown to the visitor
service_account	if using Approach B

7. Developer Notes

- Always store/send times in **UTC**; convert for display only.
- Required headers on every call: `Authorization: Bearer {token}`, `Content-Type: application/json`, `Prefer: outlook.timezone="UTC"`.
- Keep the app registration single-tenant; use Key Vault (or equivalent) for secrets, not env files checked into source control.